MCP Server Kubernetes + Claude AI Integration (On-Prem Setup)

Overview:

This setup integrates:

- mcp-server-kubernetes
- Claude
- Kubernetes cluster (kubeadm – 4 nodes)

Purpose:
Control Kubernetes using **natural language via AI**

Architecture:

```
Claude CLI (AI Agent)
      ↓ (stdio)
MCP Server (Node.js)
      ↓
Kubernetes Cluster (via kubeconfig)
```

Prerequisites

- Ubuntu 20.04 / 22.04 server
- Kubernetes cluster (kubeadm)
- kubectl configured
- Internet access

Step 1: Clone Repository

```
sudo su
cd ~
git clone https://github.com/Flux159/mcp-server-kubernetes.git
cd mcp-server-kubernetes
```

Step 2: Install Bun

```
curl -fsSL https://bun.sh/install | bash
source ~/.bashrc
bun –version
```

Install Node: above 20.X.X version

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
apt install nodejs -y
```

Step 3: Install Dependencies

```
bun install
```

If slow:

```
export BUN_INSTALL_CACHE_DIR=/tmp/bun-cache
```

Step 4: Build Project

bun run build

This generates: $ tsc && shx chmod +x dist/*.js

Step 5: Configure Kubernetes Access (if not)

```
mkdir -p ~/.kube
sudo cp /etc/kubernetes/admin.conf ~/.kube/config
sudo chown $USER:$USER ~/.kube/config
kubectl get nodes
```

Step 6: Install Claude CLI

```
curl -fsSL https://claude.ai/install.sh | bash
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
claude –help
```

Step 7: Login (IMPORTANT – On-Prem)

Use API Key method (recommended)

Create API Key

Open in browser:

https://console.anthropic.com

Generate API key

Login via CLI

claude login OR claude

Select:

2. Anthropic Console account (API usage)

paste that API key here , you will get succuessfully login prompt

Step 8: Configure MCP Server (Do it on for clouad Desktop only)

claude config edit

```
{
  "mcpServers": {
    "kubernetes": {
      "command": "node",
      "args": ["/home/<your-user>/mcp-server-kubernetes/dist/index.js"],
      "env": {
        "KUBECONFIG": "/home/<your-user>/.kube/config"
      }
    }
  }
}
```

Replace `<your-user>` accordingly

Step 9: Start Claude

claude

Step 10: Test Integration

List all Kubernetes pods

Show all nodes in my cluster

Example Prompt (Nginx Deployment):

Create a complete Kubernetes setup in my cluster with the following requirements:
1. Create a namespace called "prod"
2. Create a PersistentVolume (PV):
   - Use local storage
   - Path: /tmp/nginxlog
   - Capacity: 1Gi
   - Access mode: ReadWriteOnce
3. Create a PersistentVolumeClaim (PVC):
   - Namespace: prod
   - Request: 1Gi
   - Access mode: ReadWriteOnce
4. Create a ConfigMap:
   - Namespace: prod
   - Name: nginx-html
   - Add custom index.html content like "Welcome to Production Nginx"
5. Create an Nginx Deployment:
   - Namespace: prod
   - Use nginx:latest image
   - Mount PVC to /var/log/nginx
   - Mount ConfigMap to /usr/share/nginx/html
6. Expose the deployment using a NodePort service Add Ssl for webiste create ssl direcrtoy in same folder..
7. Ensure all resources are properly linked and working..
After creating, verify all resources and show me:
- pods – pv – pvc - services

procced with yes only.

Verify Resources
kubectl get all -n prod
kubectl get pv
kubectl get pvc -n prod

Activities Terminal Mar 25 18:54

✳ Delete production namespace setup

root@ubuntu24: /usr/local/share... root@minion1: /usr/local/share/... ✳ Delete production namespace ... root@mongo1: /home/admin1/p...

```
    kubectl get pods -n prod -o wide && \…)
═══ PODS ═══
    NAME                    READY   STATUS    RESTARTS   AGE   IP               NODE     NOMINATED NODE   READINESS GATES
    nginx-7f54bd6479-qclt5   1/1     Running   0          49s   10.233.109.153   mongo3   <none>           <none>
    +21 lines (ctrl+o to expand)

● Step 5: Test HTTPS access

● Bash(curl -sk https://10.13.10.105:30443 | grep -o '<h1>.*</h1>\|<p>.*</p>')
  <h1>Welcome to Production Nginx</h1>
    <p>Namespace: <strong>prod</strong>  |  Secured with TLS  |  Logs on PV</p>

● Everything is up and working. Here's the full summary:

  ---
  Resources Created
```

Resource	Name	Status
Namespace	prod	Active
PersistentVolume	nginx-logs-pv	**Bound** (hostPath: /tmp/nginxlog)
PersistentVolumeClaim	nginx-logs-pvc	**Bound**
ConfigMap	nginx-html	Active (index.html + nginx.conf)
Secret	nginx-tls	Active (self-signed TLS cert)
Deployment	nginx	1/1 Running
Service	nginx	NodePort 80:30080, 443:30443

```
  Files

  /home/admin1/pooja-mcp/
  ├── nginx-prod.yaml        ← all-in-one manifest
  └── ssl/
      ├── tls.crt            ← self-signed certificate (365 days)
      └── tls.key            ← private key

  Access URLs

  https://10.13.10.105:30443   ← control-plane
  https://10.13.10.215:30443   ← worker (mongo2)
  https://10.13.10.220:30443   ← worker (mongo3)
  Use -k flag with curl or accept the browser warning (self-signed cert). HTTP on port 30080 auto-redirects to HTTPS.
```

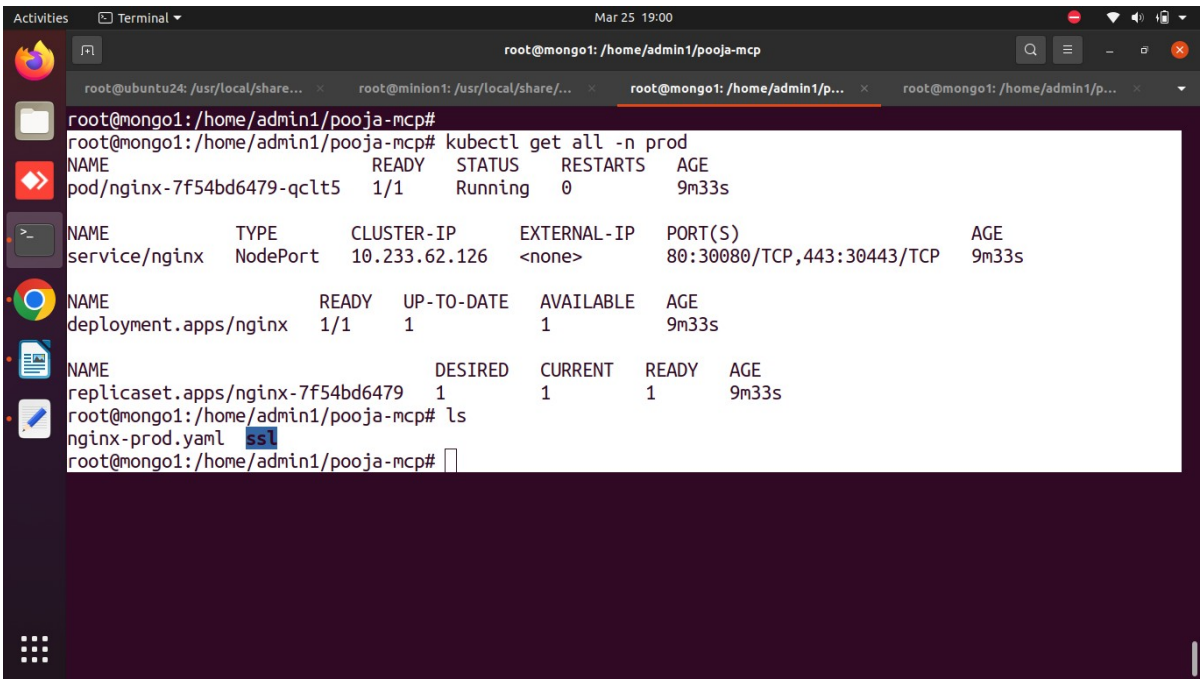
Activities Google Chrome Mar 25 18:53

Not secure https://10.13.10.215:30443

All Bookmarks

Welcome to Production Nginx

Namespace: **prod** | Secured with TLS | Logs on PV



Security Best Practice

Avoid using admin.conf in production.

Create limited access:

kubectl create serviceaccount mcp-user

kubectl create clusterrolebinding mcp-binding \

  --clusterrole=view \

  --serviceaccount=default:mcp-user

Production Improvements

- Use Nginx as reverse proxy
- Add HTTPS (Let’s Encrypt)
- Use restricted RBAC
- Monitor via Prometheus/Grafana

Key Concepts

Component	Role
Claude	AI agent
MCP Server	Middleware
Kubernetes	Execution layer

Permission error:

sudo chown -R $USER:$USER ~/mcp-server-kubernetes